

Project 3: A Bite of Peer-to-Peer

Zophia Kimberly Laud [029783219],

Ivy Le [030215976],

Mai Mai [030276114]

Instructor: Hailu Xu

California State University, Long Beach

College of Engineering

CECS 327, Sec 02, Spring 2026

April 10, 2026

Links:

Video - [<https://youtu.be/usOyWDcivTA>]

Task 1: Setup

For the P2P network setup, we developed a system where each node acts as both a client and a server, enabling decentralized communication. We also implemented a bootstrap node to handle the registration and discovery of containerized nodes.

Each node was programmed to assign itself a unique identifier using Python's uuid module and start a minimal HTTP server by using Flask. Using this, we can listen to incoming peer requests. We also created a Dockerfile to containerize the peer logic.

We then used a docker-compose.yml file to orchestrate the network. By using configurations, we were able to successfully scale the environment to many different numbers of nodes that would automatically register with the bootstrap node upon its initialization.

```
node21    Up 5 minutes    0.0.0.0:5021->5000/tcp, [::]:5021->5000/tcp
node20    Up 17 minutes   0.0.0.0:5020->5000/tcp, [::]:5020->5000/tcp
node19    Up 17 minutes   0.0.0.0:5019->5000/tcp, [::]:5019->5000/tcp
node18    Up 17 minutes   0.0.0.0:5018->5000/tcp, [::]:5018->5000/tcp
node17    Up 17 minutes   0.0.0.0:5017->5000/tcp, [::]:5017->5000/tcp
node16    Up 17 minutes   0.0.0.0:5016->5000/tcp, [::]:5016->5000/tcp
node15    Up 17 minutes   0.0.0.0:5015->5000/tcp, [::]:5015->5000/tcp
node14    Up 17 minutes   0.0.0.0:5014->5000/tcp, [::]:5014->5000/tcp
node13    Up 17 minutes   0.0.0.0:5013->5000/tcp, [::]:5013->5000/tcp
node12    Up 17 minutes   0.0.0.0:5012->5000/tcp, [::]:5012->5000/tcp
node11    Up 17 minutes   0.0.0.0:5011->5000/tcp, [::]:5011->5000/tcp
node10    Up 17 minutes   0.0.0.0:5010->5000/tcp, [::]:5010->5000/tcp
node9     Up 17 minutes   0.0.0.0:5009->5000/tcp, [::]:5009->5000/tcp
node8     Up 17 minutes   0.0.0.0:5008->5000/tcp, [::]:5008->5000/tcp
node7     Up 17 minutes   0.0.0.0:5007->5000/tcp, [::]:5007->5000/tcp
node6     Up 17 minutes   0.0.0.0:5006->5000/tcp, [::]:5006->5000/tcp
node5     Up 17 minutes   0.0.0.0:5005->5000/tcp, [::]:5005->5000/tcp
node4     Up 17 minutes   0.0.0.0:5004->5000/tcp, [::]:5004->5000/tcp
node3     Up 17 minutes   0.0.0.0:5003->5000/tcp, [::]:5003->5000/tcp
node2     Up 17 minutes   0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp
node1     Up 17 minutes   0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp
● (base) mai@mai-book p2p-system % docker ps | grep -E "bootstrap|node" | wc -l
50
● (base) mai@mai-book p2p-system % docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" | grep -E "bootstrap|node"
node50    Up 5 minutes    0.0.0.0:5050->5000/tcp, [::]:5050->5000/tcp
node49    Up 5 minutes    0.0.0.0:5049->5000/tcp, [::]:5049->5000/tcp
node48    Up 5 minutes    0.0.0.0:5048->5000/tcp, [::]:5048->5000/tcp
node47    Up 5 minutes    0.0.0.0:5047->5000/tcp, [::]:5047->5000/tcp
node46    Up 5 minutes    0.0.0.0:5046->5000/tcp, [::]:5046->5000/tcp
node45    Up 5 minutes    0.0.0.0:5045->5000/tcp, [::]:5045->5000/tcp
```

Task 2: Peer Discovery and Messaging

In this task, we implemented the discovery mechanism where nodes query the bootstrap node to retrieve a list of active peers. This allows any node to send a message to any other peer inside the network without the need for a central server for the actual data transfer.

We verified this by triggering a message from one node and observing the logs of the recipient node. The bootstrap node correctly maintained the registry, updating the peer list as nodes joined or left the network. We monitored the traffic using container logs to ensure the Flask routes were functioning as intended.

```
Sending from node30 to node2  
{"status":"received"}  
  
Sending from node30 to node48  
{"status":"received"}  
  
Sending from node3 to node8  
{"status":"received"}  
  
Sending from node40 to node25  
{"status":"received"}  
  
Sending from node50 to node36  
{"status":"received"}  
  
Sending from node47 to node6  
{"status":"received"}  
  
Sending from node22 to node20  
{"status":"received"}  
  
Sending from node30 to node29  
{"status":"received"}  
  
Sending from node2 to node1  
{"status":"received"}
```

Contributions

Mai Mai - P2P node architecture, Flask API implementation, peer registration logic, and testing.

Ivy Le - Bootstrap node development, peer discovery mechanism, and Docker Compose scaling configuration.

Zophia Kimberly Laud - UUID integration, multi-node communication validation, and report preparation.

Challenges

1. When scaling the network to a large volume of nodes, we found that the bootstrap node originally would fail, but inconsistently, to process all registration requests, most likely due to the simultaneous traffic.
 - a. We realized that staggering the container startup was necessary, so we implemented a randomized delay in the Python script to spread the registration load over several seconds.
2. Debugging the interaction between such a large number of nodes made the terminal logs move too quickly to read.
 - a. We temporarily utilized docker compose configurations to isolate the registry activity and verify that each node was being assigned its unique UUID correctly.

References

<https://flask.palletsprojects.com/>

<https://docs.python.org/3/library/uuid.html>

<https://docs.docker.com/compose/how-tos/networking/>